ENUNCIADO REEMPLAZADO POR CONFIDENCIALIDAD. ESTE DOCUMENTO NO ES EL ENUNCIADO ORIGINAL DE LA PRUEBA TÉCNICA: ES UNA REESCRITURA PROPIA QUE CONSERVA EL CONTEXTO, LOS REQUISITOS Y LOS CRITERIOS DE EVALUACIÓN, Y SE PUBLICA ASÍ POR MOTIVOS DE CONFIDENCIALIDAD.

Desafío técnico: wallet de credenciales verificables para socios de un club

La numeración de las secciones se mantiene estable porque el código, los tests y los ADRs del repositorio la citan (por ejemplo, "sección 4.1.2").

Contenido

- 1. Propósito
- 2. Cómo se va a evaluar
  - 2.1 Aspectos evaluados
  - 2.2 Fuera de la evaluación
- 3. Qué hay que entregar
- 4. El sistema
  - 4.1 Requisitos funcionales
    - 4.1.1 Casos de uso del administrador
    - 4.1.2 La credencial
    - 4.1.3 Pantalla de alta
  - 4.2 Requisitos técnicos
    - 4.2.1 Stack y arquitectura

1. Propósito

Un club de fútbol quiere entregarles a sus socios credenciales digitales que cualquiera pueda comprobar como auténticas, porque van firmadas criptográficamente. El desafío es construir el sistema que las emite y las administra.

El punto central del diseño es la división de responsabilidades: el club, que es quien tiene la relación con el socio, actúa como **Tenant** (identificado como `club-futbol`), y la emisión y la firma de las credenciales le corresponden a otro rol, el **Issuer**. Esa división tiene que verse como dos servicios distintos dentro de la solución.

2. Cómo se va a evaluar

2.1 Aspectos evaluados

Que funcione de punta a punta. En concreto:

- **UC01 (alta):** se cargan nombre, apellido, DNI, categoría y foto del socio (ver 4.1.3), el backend construye el sujeto de la credencial, el Issuer la emite y la firma, y la VC entera queda guardada.
- **Campos automáticos:** los identificadores, el período de vigencia y la firma (ver 4.1.2) se generan bien y siempre de la misma manera.
- **UC02 (listado):** las credenciales guardadas se leen y se muestran; cuando todavía no hay ninguna, la pantalla lo indica con un estado vacío.
- **Errores:** una falla de firma en el Issuer no puede dejar ninguna credencial guardada.

Además del funcionamiento:

Aspecto	Qué se espera
Diseño	La arquitectura de la solución está documentada.
Código	Se lee con facilidad y sigue buenas prácticas.
Decisiones	Todo lo que este documento deja abierto y resulte relevante se resuelve y se deja por escrito.
Interfaz	Alcanza con que sea usable: un estilo sobrio y mínimo es suficiente.

2.2 Fuera de la evaluación

No hace falta resolver (ni se va a puntuar):

- autenticación ni autorización de usuarios;
- verificar credenciales;
- revocarlas o cambiarles el estado;
- ajustarse al detalle a las especificaciones W3C de VC y DID;
- paginar el listado (filtrarlo es un agregado opcional).

3. Qué hay que entregar

1. Un repositorio Git que conserve la historia de commits.
2. Un `README.md` que explique cómo poner en marcha el proyecto: con Docker, sin Docker, o con una mezcla (algunas piezas en contenedores y otras corriendo en la máquina).

4. El sistema

4.1 Requisitos funcionales

4.1.1 Casos de uso del administrador

Hay dos casos de uso, ambos del lado del administrador del tenant. Se describen en el estilo informal de Cockburn.

UC01 — Emitir la credencial de un socio

Elemento	Detalle
Actor	Administrador del tenant
Antes	Puede entrar al formulario de alta
Después	La credencial está guardada y aparece en el listado

Flujo normal:

1. El administrador entra al formulario de alta.
2. Carga los datos del socio (ver 4.1.3), categoría incluida.
3. Envía el formulario.
4. El backend construye el `credentialSubject` y le pide la emisión al Issuer, que corre dentro del mismo backend.
5. El Issuer completa `id`, `type`, `issuer`, `validFrom`, `validUntil` y `proof` y responde con la VC ya firmada.
6. El backend guarda la credencial y confirma la operación.

Alternativa:

- **5a.** La firma falla en el Issuer: no se guarda absolutamente nada y el administrador ve el error.

UC02 — Ver las credenciales emitidas

Elemento	Detalle
Actor	Administrador del tenant
Antes	Puede entrar a la pantalla del listado
Después	Ve las credenciales de su tenant

Flujo normal:

1. El administrador entra al listado.
2. El backend busca las credenciales del tenant.
3. Por cada credencial se ven la foto, el nombre y el apellido, la categoría, el número de socio, el período de vigencia y el estado.

Alternativa:

- **2a.** Todavía no se emitió ninguna: se muestra un estado vacío.

4.1.2 La credencial

A continuación se detalla cada campo de la credencial verificable (VC): qué representa, de qué tipo es, qué parte del sistema lo produce y cómo se calcula su valor.

Campos que agrega el Issuer

Campo	Qué representa	Tipo	Cómo se obtiene
`id`	El documento emitido (esta credencial puntual, no la persona)	string (URI)	Un `Guid` nuevo en cada emisión, expresado como URI absoluta dentro del dominio del issuer: `https://credenciales.futbol.com.ar/{Guid}`, por ejemplo `https://credenciales.futbol.com.ar/8f14e45f-ceea-467e-9de1-93f5a5f4bfae`
`type`	La clase de documento según el modelo de credenciales verificables	array	Siempre `["VerifiableCredential","SocioCredential"]`
`validFrom`	Inicio de la vigencia, es decir, el momento de emisión	datetime	Instante de la emisión, en UTC
`validUntil`	Fin de la vigencia	datetime	Un año después de `validFrom`
`proof`	La firma que permite detectar si la credencial se modificó después de emitida	object	Firma simulada con HMAC-SHA256; la clave viene de una variable de entorno o de un secret. Detalle más abajo

Campos que vienen de otro lado

Campo	Qué representa	Tipo	Origen	Cómo se obtiene
`issuer`	La identidad digital del emisor (el club)	string (DID)	Configuración	Valor fijo `did:example:futbol`, tomado de una variable de entorno
`credentialSubject`	La información del socio titular	object	Tenant	Ver la tabla siguiente
`credentialStatus`	Si la credencial está activa, revocada o suspendida	integer	Tenant	`0` = active (valor inicial), `1` = revoked, `2` = suspended

Dentro de `credentialSubject`

Subcampo	Qué representa	Tipo	Origen	Cómo se obtiene
`nombre`	Nombre de pila	string	Formulario	—

Subcampo	Qué representa	Tipo	Origen	Cómo se obtiene
`apellido`	Apellido	string	Formulario	—
`dni`	Número de DNI	string	Formulario	—
`categoria`	Tipo de membresía en el club	string	Formulario	`adulto`, `juvenil` o `niño`
`foto`	Imagen para reconocer al socio	string (URI)	Formulario	Como mínimo, una URL; subir el archivo es un extra opcional
`id`	Identificador técnico y permanente del socio en su rol de sujeto (no confundir con el número de socio)	string (DID)	Tenant	`did:example:{Guid}`; se crea una sola vez y se guarda
`numeroSocio`	El número con el que el club y el socio se refieren a la membresía en la práctica	string	Tenant	Correlativo, guardado y estable aunque el sistema se reinicie; la estrategia es libre pero tiene que quedar documentada

Dentro de `proof`

Subcampo	Qué representa	Tipo	Cómo se obtiene
`type`	Algoritmo de firma	string	Siempre `"HMAC-SHA256"`
`created`	Momento de la firma	datetime	Instante de la firma en UTC; en los hechos es el mismo que `validFrom`
`verificationMethod`	A qué clave remite la firma (útil si más adelante se agrega la verificación)	string (DID)	Señala la clave de firma, por ejemplo `did:example:futbol#key-1`
`proofValue`	La firma propiamente dicha	string	Resultado del HMAC-SHA256 pasado a base64

Cálculo de `proofValue`

La forma canónica de la credencial no se deja a elección: está definida acá para que no haya dos maneras válidas de serializar lo mismo. El Issuer toma la credencial completa con todos los campos de 4.1.2 salvo `proof` y la escribe como JSON respetando lo siguiente:

1. **Formato compacto:** ni espacios ni saltos de línea.
2. **Orden del primer nivel**, alfabético: `credentialStatus`, `credentialSubject`, `id`, `issuer`, `type`, `validFrom`, `validUntil`.
3. **Orden dentro de `credentialSubject`**, también alfabético: `apellido`, `categoria`, `dni`, `foto`, `id`, `nombre`, `numeroSocio`.
4. **Fechas:** ISO-8601 en UTC, al segundo, terminadas en `Z` y sin fracción de segundo (por ejemplo `2026-08-09T14:32:10Z`).
5. **Caracteres no ASCII** (acentos, ñ): se escriben tal cual en UTF-8, nunca como secuencias `\uXXXX`. Ojo con .NET: `System.Text.Json` escapa esos caracteres si no se le configura otro encoder, y ese descuido

alcanza para que el hash deje de ser reproducible.

Para una credencial de muestra, el JSON canónico (lo que entra al HMAC) es:

```
{"credentialStatus":0,"credentialSubject":{"apellido":"Pérez","categoria":"adulto","dni":"30123456","foto":"https://cdn.futbol.com.ar/socios/8f14e45f.jpg","id":"did:example:3fa85f64-5717-4562-b3fc-2c963f66afa6","nombre":"Juan","numeroSocio":"000123"},"id":"https://credenciales.futbol.com.ar/8f14e45f-ceea-467e-9de1-93f5a5f4bfae","issuer":"did:example:futbol","type":["VerifiableCredential","SocioCredential"],"validFrom":"2026-08-09T14:32:10Z","validUntil":"2027-08-09T14:32:10Z"}
```

Con ese texto se calcula `HMAC-SHA256(json_canónico, clave_secreta)`, donde la clave sale de una variable de entorno o de un secret y jamás figura en el código. Los bytes resultantes se pasan a base64 y ese es el `proofValue`.

Se trata de una firma de juguete: no pretende ser un mecanismo real de firma digital como JWS o Linked Data Proofs, solo representar que la credencial lleva una prueba criptográfica. Verificarla queda afuera del alcance (ver 2.2).

Un `proof` de ejemplo:

```
"proof": {
  "type": "HMAC-SHA256",
  "created": "2026-08-09T14:32:10Z",
  "verificationMethod": "did:example:futbol#key-1",
  "proofValue": "b8f3a1e9c02d4f7a6e1b9c3d8f2a5e7b1c4d6f9a0e3b7c1d5f8a2e6b9c0d3f7a"
}
```

Qué se guarda y qué se muestra

- **Se guarda** la VC entera, tal como se describe en esta sección, siguiendo el modelo de datos de 4.2.1.
- **Se muestra** en el listado (UC02) solamente lo necesario: el `credentialSubject`, la vigencia y el estado.
- **Es opcional mostrar** los campos propios del protocolo (`id`, `type`, `issuer`, `proof`); si se incluyen, puede ser en un detalle que se despliega.

4.1.3 Pantalla de alta

El formulario pide solo cinco datos:

Dato	Tipo de control
Nombre	Campo de texto
Apellido	Campo de texto

Dato	Tipo de control
DNI	Campo de texto que acepte solo números
Categoría	Lista desplegable con los valores definidos en 4.1.2
Foto	Campo de URL como mínimo; subir un archivo es opcional

Todo lo demás lo completa el sistema: el `id` de la VC, `credentialSubject.id`, `numeroSocio`, `validFrom`, `validUntil`, `credentialStatus` y `proof`. Ninguno de esos valores se puede editar en el formulario; tienen que aparecer después del alta, como parte del resultado.

Como sugerencia, al terminar el alta conviene mostrar una pantalla de confirmación que informe qué número de socio se asignó y hasta cuándo vale la credencial, y desde ahí volver al listado.

4.2 Requisitos técnicos

4.2.1 Stack y arquitectura

Backend e Issuer

- El backend se hace en .NET, en su última versión estable.
- Expone **un solo controller** (por ejemplo `CredentialsController`). Ese controller recibe las altas, arma el `credentialSubject`, llama al Issuer, guarda la credencial y ofrece el listado.
- El **Issuer es un servicio in-process**: vive dentro del backend y no publica ningún endpoint. Recibe el `credentialSubject` y le suma `id`, `type`, `issuer`, `validFrom`, `validUntil` y `proof`.
- **Clean Architecture no es obligatoria.** Cómo se reparten las capas y los proyectos lo decide quien resuelve el desafío.

Interfaz

- La tecnología y el aspecto visual son libres; Angular, en su última versión estable, es la alternativa preferida.
- Sea cual sea la elección, tiene que tener las pantallas de alta y de listado.

Persistencia

- **El motor es a elección** (PostgreSQL, SQL Server, SQLite, Redis, archivos en disco o cualquier otro), y la elección tiene que quedar documentada.
- La forma de modelar los datos (tablas, colecciones, claves, archivos, según el motor) también es libre, con una condición mínima: tiene que haber una entidad `credentials` donde se guarden las credenciales emitidas.

Configuración y ejecución

- Toda la configuración entra por variables de entorno.
- Contenerizar el backend y el frontend es opcional. En su lugar se puede documentar en el `README.md` cómo correr cada uno en local (por ejemplo, `dotnet run` y el comando de arranque que corresponda al framework de UI).

- Si el motor elegido corre como un servicio aparte (PostgreSQL, SQL Server, Redis), se recomienda sumarlo a un `docker-compose.yml` para que todo el entorno se levante fácil. Los motores embebidos o basados en archivos (SQLite, disco) no lo necesitan.